



Aplicación Web CRUD — Sistema de Reservas B2B

Fase 4 — Desarrollo de Frontend
(Unidad 5)

Asignatura: Aplicaciones Web II

Docente / Asesor: _____

Integrante(s): _____

Fecha de entrega: ____ / ____ / ____

Informe de avance — Fase 4

1. Resumen

En la Fase 3 el backend quedó operando la funcionalidad completa de manipulación de datos (autenticación, sucursales, mesas y reservas) sobre una base de datos PostgreSQL, con acceso a datos mediante el ORM Prisma y sus servicios listos para ser consumidos. La Fase 4 construye el **entramado de frontend** que consume esos servicios por HTTP, de modo que un usuario real ejecuta el ciclo CRUD completo (Altas, Bajas, Cambios y Consultas) desde la interfaz, y añade **pruebas de humo** que verifican la integración entre frontend y backend de extremo a extremo.

El frontend es una aplicación **Next.js 16 (App Router) + React 19 + Tailwind CSS 4**. Toda la comunicación con el backend se centraliza en un único cliente HTTP (`frontend/src/lib/api.ts`) y la sesión se gestiona con un token JWT almacenado en el navegador.

2. Entramado de frontend y consumo de servicios

2.1 Arquitectura de consumo

```
Navegador
|
|-- Componentes de página (app/**/page.tsx)      ("use client")
|     llaman a
|     v
|-- Cliente API (lib/api.ts)
|     fetch() con Authorization: Bearer <JWT>
|     - inyecta el token desde localStorage
|     - normaliza los errores en ApiError { mensaje, status }
|     - ante 401 (token ausente / expirado): limpia la sesión y va a
/login
|     v
|-- Backend Express (http://localhost:4000)
|     /auth      /branches      /reservations
```

Elemento	Responsabilidad
<code>lib/session.ts</code>	Store de sesión (token + usuario) sobre <code>localStorage</code> , expuesto a React con <code>useSyncExternalStore</code> ; se sincroniza entre pestañas con el evento <code>storage</code> .
<code>lib/useSession.ts</code>	Hook de guarda: expone <code>{ usuario, estado }</code> y redirige a <code>/login</code> si no hay sesión, o a <code>/reservas</code> si el rol no está

	autorizado.
components/NavBar.tsx	Navegación reactiva: muestra nombre y rol, y sólo los enlaces permitidos (p. ej. <i>Sucursales</i> únicamente para el rol ADMIN).
lib/api.ts (intercepción 401)	Si el backend responde 401, se limpia la sesión y se fuerza el regreso a /login?expirada=1.

2.2 Mapa de pantalla → servicio de backend

Pantalla (ruta)	Acción del usuario	Método lib/api.ts	Endpoint backend
/registro	Alta de cuenta	authApi.register	POST /auth/register
/login	Inicio de sesión	authApi.login	POST /auth/login
/perfil	Ver identidad	authApi.me	GET /auth/me
/perfil	Cambiar contraseña	authApi.changePassword	PATCH /auth/password
/reservas	Consulta (Leer)	reservationsApi.list	GET /reservations
/reservas	Confirmar reserva	reservationsApi.confirm	PATCH /reservations/:id
/reservas/nueva	Listar sucursales	branchesApi.list	GET /branches
/reservas/nueva	Consultar disponibilidad	branchesApi.availability	GET /branches/:id/availability
/reservas/nueva	Alta de reserva (Crear)	reservationsApi.create	POST /reservations
/reservas/:id/editar	Cargar reserva	reservationsApi.getOne	GET /reservations/:id
/reservas/:id/editar	Cambio (Actualizar)	reservationsApi.update	PATCH /reservations/:id
/reservas/:id/cancelar	Baja (borrado lógico)	reservationsApi.cancel	DELETE /reservations/:id
/sucursales	Consulta / Alta de sucursal	branchesApi.list / create	GET / POST /branches
/sucursales/:id	Detalle / Cambio / Baja	branchesApi.getOne / update / remove	GET / PATCH / DELETE /branches/:id
/sucursales/:id	Alta / Baja de mesa	branchesApi.addMesa / removeMesa	POST / DELETE /branches/:id/mesas

Con esto quedan cubiertas en la interfaz las cuatro operaciones del CRUD sobre las dos entidades de negocio: **reservas** y **sucursales / mesas**.

2.3 Ajustes de backend para el consumo

1. **GET /auth/me** — devuelve `{ id, nombre, email, rol, sucursalId }` a partir del token; sustituye la lectura de datos cacheados en el navegador.
2. **Reservas con relaciones** — el repositorio incluye `sucursal` y `mesa` en las respuestas, para mostrar «Sucursal Centro · Mesa 3» en lugar de identificadores.
3. **Esquemas de validación** — los identificadores pasan de `uuid()` a «cadena no vacía», admitiendo tanto los UUID generados por Prisma como los IDs legibles del *seed* de demostración.
4. **Borrado en cascada** — al eliminar una sucursal se eliminan sus mesas; una sucursal con reservas asociadas sigue protegida y devuelve un error 409 con mensaje claro.

3. Justificación: REST y no WebSockets

Los lineamientos piden justificar el empleo de *websockets* en caso de utilizarlos. **Este proyecto no los utiliza**; toda la comunicación es **HTTP/REST de tipo petición-respuesta**. Las razones son:

- Las operaciones son **transaccionales y las inicia el usuario** (crear, editar o cancelar una reserva; administrar sucursales). No hay un flujo continuo de datos que justifique una conexión persistente.
- **No hay colaboración en tiempo real** entre varios usuarios sobre el mismo recurso ni necesidad de *push* del servidor al cliente. La disponibilidad de mesas se consulta **bajo demanda** al abrir la pantalla de nueva reserva.
- REST sobre HTTP es **más simple de operar, cachear, depurar y escalar** (el servidor no mantiene estado de conexión) y encaja con el modelo CRUD del proyecto.
- Las notificaciones al cliente (confirmación, cancelación, reprogramación) se resuelven por **correo electrónico** desde el backend, no por un canal en vivo.

Si en el futuro se requiriera un tablero de ocupación de mesas actualizado al instante para el *host*, entonces sí se evaluaría WebSockets o *Server-Sent Events* para esa vista concreta.

4. Pruebas de humo de la integración frontend ↔ backend

Se implementaron **dos niveles** de prueba de humo, ambos ejecutados contra el backend y la base de datos reales.

4.1 Nivel API — backend/src/tests/http-smoke.test.ts

Levanta la aplicación Express en un puerto efímero y ejercita **por HTTP** los mismos endpoints, y en el mismo orden, que consume `frontend/src/lib/api.ts`. Es la evidencia de que «los servicios están listos para ser consumidos por el frontend»: registro → login → `/auth/me` → alta de sucursal y mesa → disponibilidad → alta / consulta / cambio / confirmación / baja de reserva → cambio de contraseña → bloqueo sin token.

Comando:

```
cd backend
npm test                # Fase 3 (servicios/ORM) + Fase 4 (integración HTTP)
npm run test:integration # sólo la prueba de integración frontend↔backend
```

Resultado obtenido:

```
ok 1 - salud del backend
ok 2 - auth: registro, login y perfil (/auth/me)
ok 3 - sucursales: alta de sucursal y mesa, listado y disponibilidad
ok 4 - reservas: alta, consulta con relaciones, cambio, confirmación y
baja
ok 5 - auth: cambio de contraseña y bloqueo sin token
ok 6 - auth: registro, login y cambio de contraseña
ok 7 - sucursales: alta, consulta, cambio y disponibilidad
ok 8 - reservas: alta, consulta, cambio y baja (cancelación)
1..8
# tests 8
# pass 8
# fail 0
```

[CAPTURA 1]

Terminal ejecutando `npm test` en la carpeta `backend`, mostrando las 8 pruebas en verde (# pass 8 / # fail 0).

4.2 Nivel E2E (navegador) — frontend/e2e/reservas.spec.ts

Pruebas con **Playwright** que abren un navegador real y validan el recorrido del usuario a través de la interfaz. Antes de la suite, `globalSetup` reseeda la base de datos (estado conocido) y el `webServer` de Playwright levanta el backend y el frontend.

Prueba	Qué valida
--------	------------

Un cliente crea, edita y cancela una reserva desde la UI	CRUD completo de reserva desde el navegador: selección de sucursal, consulta de disponibilidad, alta, verificación en el libro, cambio de personas y cancelación (estado CANCELADA).
Sin sesión, /reservas redirige a /login	La guarda de sesión del frontend funciona.
Un admin da de alta una sucursal y una mesa	CRUD de sucursales / mesas desde la UI con rol ADMIN.

Comando (requiere PostgreSQL disponible, p. ej. el contenedor `reservas-pg`):

```
cd frontend
npx playwright install chromium # sólo la primera vez
npm run test:e2e
```

Resultado obtenido:

Running 3 tests using 1 worker

```
ok 1 un cliente crea, edita y cancela una reserva desde la UI (2.4s)
ok 2 sin sesión, /reservas redirige a /login (0.3s)
ok 3 un admin da de alta una sucursal y una mesa (1.3s)

3 passed (8.2s)
```

[CAPTURA 2]

Terminal ejecutando `npm run test:e2e` en la carpeta `frontend`, mostrando las 3 pruebas E2E en verde (3 passed).

[CAPTURA 3 — opcional]

Reporte HTML de Playwright (`npx playwright show-report`) con las 3 pruebas pasando.

5. Guía para reproducir el entorno y generar las capturas

5.1 Base de datos (PostgreSQL vía Docker)

```
docker run -d --name reservas-pg \  
  -e POSTGRES_USER=usuario -e POSTGRES_PASSWORD=password -e \  
  POSTGRES_DB=reservas_b2b \  
  -p 5432:5432 postgres:16-alpine
```

5.2 Backend

```
cd backend  
cp .env.example .env  
npm install  
npx prisma migrate deploy      # aplica el esquema  
npm run prisma:seed            # datos de demostración  
npm run dev                    # queda en http://localhost:4000
```

Usuarios de demostración (contraseña `clave1234`):

Correo	Rol
admin@demo.test	ADMIN
host@demo.test	HOST (Sucursal Centro)
cliente@demo.test	CLIENTE

5.3 Frontend

```
cd frontend  
cp .env.local.example .env.local      #  
NEXT_PUBLIC_API_URL=http://localhost:4000  
npm install  
npm run dev                          # queda en http://localhost:3000
```

5.4 Secuencia sugerida para las capturas del Anexo

1. Abrir `http://localhost:3000` → capturar la pantalla de inicio.
2. Ir a **/login**, entrar como `cliente@demo.test` → capturar el formulario.
3. Ir a **/reservas/nueva**, elegir sucursal, pulsar «Ver disponibilidad», elegir un horario → capturar la grilla de slots.
4. Pulsar «Reservar» → en **/reservas** capturar el libro con la reserva PENDIENTE (muestra sucursal y mesa).
5. Entrar a **/perfil** → capturar identidad real (desde `/auth/me`) y el formulario de cambio de contraseña.
6. Cerrar sesión, entrar como `admin@demo.test`, ir a **/sucursales** → capturar el listado y el formulario de alta.

7. Abrir el detalle de una sucursal → capturar la edición y la gestión de mesas.

6. Anexo — Capturas del frontend en funcionamiento

[CAPTURA A]

Pantalla de inicio de sesión (/login).

[CAPTURA B]

Nueva reserva con disponibilidad por horarios (/reservas/nueva).

[CAPTURA C]

Libro de reservas con estado, sucursal y mesa (/reservas).

[CAPTURA D]

Edición de una reserva (/reservas/:id/editar).

[CAPTURA E]

Cancelación de una reserva (/reservas/:id/cancelar).

[CAPTURA F]

Perfil de usuario y cambio de contraseña (/perfil).

[CAPTURA G]

Administración de sucursales — listado y alta (/sucursales).

[CAPTURA H]

Detalle de sucursal — edición, baja y gestión de mesas (/sucursales/:id).

7. Repositorio

El avance está integrado en el repositorio de control de versiones: `github.com/rdelavega/b2b_reservas`, rama `main`.

Commit	Contenido
Fase 4: soporte backend para consumo de frontend	GET /auth/me; reservas con sucursal y mesa; seed de datos.
Fase 4: entramado frontend consumiendo el backend	Cliente API, sesión y guardas, NavBar, pantallas /registro y /sucursales, rediseño de /reservas/nueva, /perfil real, retiro del modo demo.
Fase 4: pruebas de humo de integración frontend-backend	Prueba HTTP + pruebas E2E de Playwright.
Fase 4: documento ejecutivo y README	Este documento y actualización del README.
Fase 4: borrado de sucursal en cascada sobre mesas	Migración y manejo de error 409.

8. Entregables de la fase (lista de verificación)

- [x] Entramado frontend con solicitudes a backend integradas.
- [x] CRUD completo consumible desde la interfaz (reservas y sucursales / mesas).
- [x] Pruebas de humo de la integración frontend ↔ backend (API + E2E) con resultados.
- [x] Documento ejecutivo con la justificación de REST.
- [x] Avance integrado en el repositorio (rama `main`).

